Synthesize and Reward — Reinforcement Learning for Multi-Step Tool Use in Live Environments

Ibrahim Abdelaziz*, Asim Munawar, Kinjal Basu, Maxwell Crouse,
Chulaka Gunasekara, Suneet Katrekar, Pavan Kapanipathi
IBM Research

Abstract

Training LLMs to orchestrate multi-step tool calls is held back by three coupled obstacles: realistic stateful execution environments are costly to build, synthetic training queries are often detached from the server's actual state (so the generated tool calls fail to execute), and recall-based RL rewards incentivize verbose tool-calling patterns. We present PROVE (Programmatic Rewards On Verified Environments), a framework with three contributions: (1) a library of 20 stateful MCP (Model Context Protocol) servers exposing 343 tools, enabling live-execution RL training with session-scoped state isolation; (2) a state-machine data synthesis pipeline that generates multi-turn tool-call trajectories *grounded in live-sampled server state*, so generated queries reference entities that actually exist; and (3) a multi-component programmatic reward with an adaptive efficiency penalty that counters the verbosity incentive of recall-based rewards. We train four models (Qwen3-4B, Qwen3-8B, Qwen2.5-7B, Granite-4.1-8B) with GRPO on the resulting $\sim$13K training examples. On BFCL Multi-Turn, τ^2-bench, and T-Eval, PROVE yields improvements of up to $+10.2$, $+6.8$, and $+6.5$ points respectively, demonstrating that this framework yields consistent gains on multi-step tool orchestration across two model families.

1 Introduction

Training LLMs to orchestrate multi-step tool calls—selecting the correct API, filling parameters from context, and respecting inter-call data dependencies—is a prerequisite for deploying autonomous agents in production [Yao et al., 2023, Schick et al., 2023]. The problem is harder than single-turn function calling: the model must propagate intermediate results, handle missing information gracefully, and abstain when no tool is applicable. Two complementary lines of work have emerged in response.

Data synthesis and supervised fine-tuning. xLAM [Zhang et al., 2024] releases a large corpus of verified single-turn function-calling traces, and APIGen-MT [Prabhakar et al., 2025] extends this to multi-turn trajectories. More recent pipelines target realism and scale for multi-turn tool use: Magnet [Yin et al., 2025] composes traces from tool-dependency graphs, ToolACE-MT [Zeng et al., 2026] introduces non-autoregressive multi-turn generation, and TOUCAN [Xu et al., 2025] scales synthesis to 1.5M traces against real MCP servers. These pipelines typically feed a separate SFT stage and are not coupled to execution feedback during training.

Reinforcement learning for tool use. A parallel line uses RL to shape tool-use behavior from reward signals without large SFT corpora. ToolRL [Chen et al., 2025], ReTool [Feng et al., 2025], StepTool [Yu et al., 2024], Tool-R1 [Zhang et al., 2025b], Nemotron-Research-Tool-N1 [Zhang et al., 2025a], MUA-RL [Zhao et al., 2025], and Tool Zero [Zeng et al., 2025] explore per-call and step-grained rewards, multi-turn user simulation, and RL-from-scratch. Closest to our setting, Cheng et al. [2026] decomposes the reward into per-call validity and sequence-level consistency on a single cached-API domain, while the concurrent AgenticQwen [Qwen Team, 2026] scales to $\sim$100K branching trajectories with LLM-as-judge rewards (Qwen3-235B).

*Correspondence: ibrahim.abdelaziz1@ibm.com

Despite this progress, three gaps remain unaddressed jointly: (i) most pipelines execute against cached or simulated APIs, missing the state-dependent failure modes that arise in live, stateful services; (ii) synthetic queries are often detached from the server's actual state, producing traces whose parameters do not correspond to any real entity; and (iii) sequence-level rewards based purely on recall incentivize *verbosity*—RL-trained models emit excess tool calls to maximize GT coverage, a regression that is rarely penalized explicitly.

We close these gaps with PROVE (Programmatic Rewards On Verified Environments),[1] a framework that tightly couples data synthesis to the RL loop through a shared live-execution backend:

1. **Live MCP environments.** A library of 20 stateful Model Context Protocol servers exposing 343 tools, with session-scoped state isolation, used for both data synthesis and RL training. Unlike cached APIs, these capture realistic state-dependent execution dynamics.

2. **Grounded state-machine data synthesis.** A pipeline that probes each live server for real entities, grounds query generation in that sampled state, drives a state-machine teacher through multi-turn conversations against the server, and replay-validates every trace. This produces $\sim$13K training examples (multi-turn MCP conversations, missing-function clarification, and abstention) without manual annotation or an LLM-as-judge.

3. **Programmatic multi-component reward.** A five-part reward decomposing tool-use quality into validity, dependency-ordered coverage, an *adaptive efficiency penalty* with a complexity-scaled call budget, a tool-name signal, and an argument-value matching bonus—fully programmatic with no external judge model (§3.3). An ablation (§4.5) shows that the adaptive budget and the tool-name signal are the two most load-bearing components.

Table 1 summarizes results across four models from two architecture families, trained with identical hyperparameters.

We evaluate on BFCL Multi-Turn [Patil et al., 2024], τ^2-bench [Barres et al., 2025], and T-Eval [Chen et al., 2023]. PROVE improves all four models on all three benchmarks, using $\sim$13K training examples, roughly $8\times$ less than concurrent RL pipelines such as AgenticQwen [Qwen Team, 2026], and no judge model. Figure 1 illustrates how the system works.

[1] Code, the 20 MCP server library (343 tools), synthesized training data, and trained model checkpoints will be released soon.

2 Related Work

Data Synthesis and Supervised Fine-Tuning for Tool Use. A growing body of work scales synthetic data to train tool-using LLMs via supervised fine-tuning. xLAM [Zhang et al., 2024] releases a large corpus of verified single-turn function-calling traces, and APIGen-MT [Prabhakar et al., 2025] extends this to multi-turn trajectories with stronger verification. Magnet [Yin et al., 2025] synthesizes multi-turn traces by translating a tool-dependency graph into natural conversations and distilling them into smaller models. ToolACE-MT [Zeng et al., 2026] proposes non-autoregressive generation for agentic multi-turn interactions, and TOUCAN [Xu et al., 2025] scales synthesis to 1.5M traces from real-world MCP servers. Closer to our setting, Crouse et al. [2026] address the challenge of simulating complex multi-turn interactions when the execution backend is stateless. These pipelines improve trajectory quality but typically feed a separate SFT stage and do not train against live execution feedback. Our pipeline differs in three ways: (i) it is driven by a state machine that grounds queries in *live* server state at generation time, (ii) it replay-validates every conversation against a freshly reset environment, and (iii) it produces $\sim$13K examples that feed directly into RL rollouts rather than an intermediate SFT corpus.

Reinforcement Learning for Tool Use. Reinforcement learning has become a popular way to elicit tool-use behavior without large SFT corpora. ToolRL [Chen et al., 2025] showed that reward signals alone can teach tool-use capabilities, and ReTool [Feng et al., 2025] combined reasoning and tool use through strategic RL. StepTool [Yu et al., 2024]

Table 1: Improvement over base model (Δ) after training with PROVE. All models trained for 350 GRPO steps.

Model	BFCL MT	Δ	TAU2	Δ	T-Eval	Δ
Qwen3-4B	**29.0**	+10.2	**28.6**	+3.6	**78.2**	+1.5
Qwen3-8B	**26.9**	+0.9	**29.3**	+3.5	**80.7**	+1.9
Qwen2.5-7B	**16.8**	+3.7	**21.8**	+6.8	**73.4**	+6.5
Granite-4.1-8B	**33.5**	+0.1	**31.3**	+6.4	**77.9**	+4.4

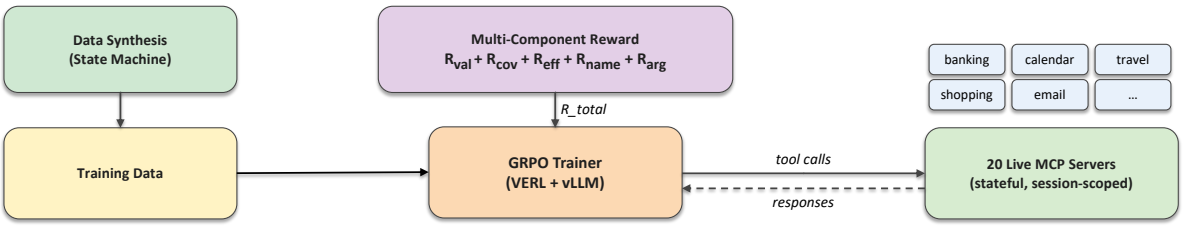

Figure 1: System overview. An LLM teacher generates multi-turn trajectories against live MCP servers using auto-discovered dependency graphs. The GRPO trainer rolls out against the same servers and computes the five-component reward. Session-scoped state isolation ensures reproducibility.

introduces step-grained rewards for multi-step tool chains, while Nemotron-Research-Tool-N1 [Zhang et al., 2025a] explores reinforced reasoning for tool-using models. Tool-R1 [Zhang et al., 2025b] targets sample efficiency with compositional multi-step rewards, and MUA-RL [Zhao et al., 2025] trains multi-turn policies against an LLM user simulator inside the RL loop. Tool Zero [Zeng et al., 2025] trains tool-using agents via pure RL from scratch without SFT warm-up. Cheng et al. [2026] propose decomposing tool-use reward into per-call validity and sequence-level consistency components on a single cached-API domain; our validity term follows this graduated formulation but replaces AST comparison with live MCP schema validation and execution. Concurrent work by Qwen Team [2026] trains tool-use agents via GRPO with behavior-tree data flywheels (∼100K examples) and LLM-as-judge rewards (Qwen3-235B). Relative to this line, PROVE differs along two axes: we train against *live stateful* environments rather than cached or simulated APIs, and our reward is *fully programmatic*—no external judge model—with an adaptive efficiency penalty that directly targets the verbosity incentive of pure-recall coverage rewards unaddressed in prior work.

3 Method

We describe the three methodological components: the live MCP environment framework (§3.1), the data synthesis pipeline (§3.2), and the multi-component reward (§3.3).

3.1 Live MCP Environment Framework

The Model Context Protocol [MCP; Anthropic, 2024] defines a standardized interface for LLMs to discover and invoke tools exposed by external servers. We build a library of 20 MCP environment servers, each running as an independent subprocess communicating over stdio. Each server exposes a domain-specific set of tools (10–40 per environment) with OpenAI-compatible function schemas. The same set of environments is used for both data synthesis and RL training.

Architecture. An `MCPManager` orchestrates server lifecycle: starting, stopping, and resetting environments. During RL training, a `MCPTool` wrapper integrates with the VERL framework [Sheng et al., 2024], routing each model-generated tool call to the appropriate MCP server and returning the execution response. Per-rollout state isolation is maintained via session-scoped state management:

each rollout episode receives a unique session ID, ensuring that tool calls from one rollout cannot contaminate another's state.

Domain Coverage. The library covers 20 environments spanning finance, productivity, commerce, travel, social, IoT, developer tools, and knowledge management. Table 2 lists the environments with their user-visible tool counts and characteristic state behavior. Every environment is stateful—account balances change after transfers, calendar events persist, filesystem trees accumulate edits—providing realistic execution dynamics that static caches cannot capture.

Advantages over Static Caches. Compared to the cached-API approach of Cheng et al. [2026], live MCP execution offers: (1) *Scalability*: adding a new domain requires only implementing an MCP server, not pre-computing a response cache; (2) *Realism*: live execution captures state-dependent failures (e.g., insufficient balance, expired coupons) that static caches miss; (3) *Determinism*: session-scoped state with deterministic reset ensures reproducible initial conditions.

3.2 Data Synthesis Pipeline

We generate training data through a *state-machine orchestrator* that drives a teacher LLM through multi-turn tool-use conversations against live MCP servers. Figure 2 illustrates the pipeline at a glance.

Step 1: Auto-Discovered Dependency Graph. For each environment, the pipeline probes all $\binom{n}{2}$ tool pairs and asks an LLM to classify each relationship as *explicit* (output of tool A is a required input of tool B), *implicit* (A must execute first to establish state), or *none*. The resulting directed graph is hashed against the tool schema and cached, so the $O(n^2)$ cost is paid once per environment. From this graph we extract length-2 to length-5 tool chains that seed realistic multi-step queries.

Step 2: Live-State Sampling (Grounded Query Generation). A naive query generator would produce queries referencing fabricated entities—account #12345, device `tv_bedroom`, user `rachel@example.com`—that crash on execution because those entities do not exist in the server's state. We avoid this with a per-environment *sampler* that probes the live MCP server *before* query generation: it calls read-only discovery tools (e.g., `get_unique_values`, `search`, `list`) to enumerate real entities, filters them to those with sufficient supporting data (e.g., cities with at least 20 businesses, accounts with non-zero balance), and caches a compact "sampling context" per environment (refreshed every k conversations). This context—real IDs, names, categories, and value ranges—is injected into the query-generation prompt along with an anti-hallucination instruction that constrains the LLM to use *only* entities present in the context. The result is queries whose arguments exist in the server's actual state, making the chosen chain of tools executable end-to-end. For chain-seeded queries, the sampler additionally returns *chain-aligned context*: a subset of entities most relevant to the specific tool chain being generated.

Step 3: State-Machine Orchestrator. Conversation generation is driven by an explicit finite state machine that tracks five groups of states—*query*, *turn*, *tool-execution*, *response*, and *continuation*—with transitions triggered by the outcome of each LLM call or tool execution. A single turn proceeds as:

1. QUERY GENERATION. A user query is sampled from the dependency-graph chains, grounded on the sampling context from Step 2, conditioned on a persona and reference date, and stratified by information level (60% complete, 20% missing-required, 20% minimal).

2. LLM PROCESSING. The teacher LLM (Gemma-4-31B-it) is prompted with the query and tool schemas. A validator checks for well-formed tool calls; invalid responses trigger a fallback regeneration.

3. TOOL EXECUTION. Tool calls are dispatched to the live MCP server. The outcome is classified as SUCCESS, PARTIAL-SUCCESS, or FAILURE, and drives the next transition.

Table 2: The 20 MCP environments used for RL rollouts. All servers are stateful with session-scoped isolation; *Tools* counts the user-visible tool surface exposed to the agent (helper/debug tools are hidden).

Category	Environment	Tools	State / characteristic behavior
Finance	banking	17	Accounts, transfers, balances; sensitive-param provenance
	trading	13	Portfolios, orders, market data; stateful order book
	payments	10	Invoices, refunds, webhooks; transactional state
Productivity	calendar	17	Events, invitees, recurring rules; time-dependent queries
	email	17	Inbox, drafts, threads, labels; append-only state
	filesystem	40	Files, dirs, permissions; deepest state (cd/ls/mv/cp chains)
Commerce	shopping	23	Catalog, cart, checkout; stateful cart
	marketplace	13	Listings, bids, seller ops; multi-entity state
	retail_chain	11	Inventory across stores; location-scoped state
Travel	travel_booking	20	Flights, hotels, itineraries; booking state
	maps	16	Geocoding, routing, POIs; mostly stateless lookups
Social / Comm.	social_media	15	Posts, follows, feeds; append + feed state
	team_chat	11	Channels, messages, threads; append-only state
IoT / Devices	iot_devices	25	Smart-home devices; sensor state + actuators
	vehicle	20	Ignition, climate, navigation; mode-gated actions
Knowledge / CRM	crm	16	Leads, contacts, deals; relational state
	issue_tracker	20	Issues, sprints, assignees; workflow transitions
	budget	12	Expenses, categories; accumulative state
Lifestyle	food_delivery	17	Menus, orders, tracking; order lifecycle state
	video_meeting	10	Meetings, participants, recordings; session state

4. RECOVERY. On failure, the state machine selects among retry-with-corrected-params, retry-with-alternative-tool, or graceful give-up with a fallback explanation.

5. CONTINUATION. A decision module samples whether to end the conversation, generate a follow-up, or ask a clarification, using a turn-decay schedule bounded by `min_turns`$= 2$ and `max_turns`$= 3$.

Step 4: Robustness Knobs. Several probabilistic perturbations are applied during generation to broaden the training distribution: (a) *Distractor injection* (40%): 3–8 unrelated tools from other environments are added to the candidate set, forcing tool discrimination; (b) *Enum stripping* (30%): enumerated value lists are removed from parameter schemas, training the model to infer constraints from descriptions; (c) *Irrelevance queries* (5%): queries that no available tool can satisfy, training refusal behavior; (d) *Missing-function*: a subset of tools is hidden after chain selection, producing abstention or clarification examples.

Step 5: Replay Validation and Deduplication. Each completed conversation is replayed against a freshly reset MCP environment. We count only schema-level and execution errors (not empty-result responses) and discard conversations with error rate above 30%. A provenance check ensures that sensitive parameters (passwords, tokens) appear only when traceable to prior user turns or tool outputs. Surviving conversations are deduplicated by Jaccard similarity on tool-call sequences (threshold 0.70).

Training Data. The final training set comprises 13,517 examples drawn from three sources: 10,895 multi-turn MCP conversations across 20 environments; 1,500 clarification trajectories generated by the missing-function variant of Step 3; and 1,122 externally-sourced abstention examples ($G = \emptyset$)—

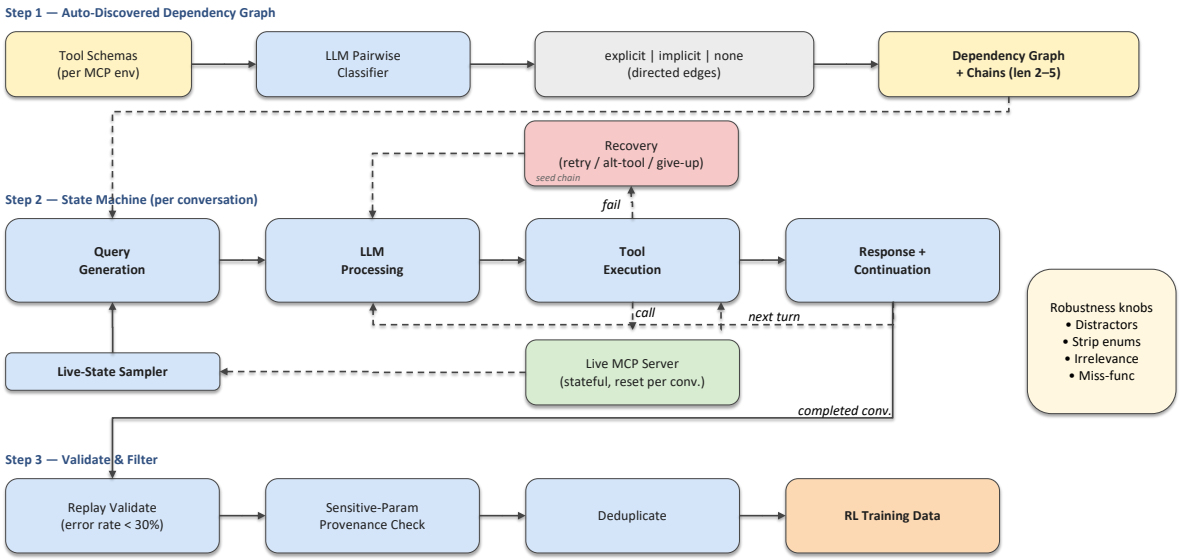

Figure 2: State-machine-driven data synthesis. Per environment, we (1) auto-discover a tool dependency graph, (2) run a state machine that alternates query generation, tool execution, and continuation decisions against a live MCP server, and (3) replay-validate each conversation before conversion to RL training data.

806 from When2Call [Ross et al., 2025] (no-tool-call scenarios) and 316 from xLAM-Irrelevance [MadeAgents, 2024] (irrelevant queries). All models are trained in a single stage for 350 GRPO steps.

3.3 Multi-Component Reward

Our reward decomposes into five components: **validity** (is each call structurally correct and executable?), **coverage** (are all required steps completed in dependency order?), **efficiency** (was it done concisely?), **tool-name guidance** (did the model select the right tools?), and **argument-value matching** (do the chosen argument *values* agree with ground truth, not just the argument keys?). The first two address natural desiderata for any tool-calling system. The efficiency penalty—specifically its complexity-scaled, adaptive budget—and the tool-name bonus are our two most load-bearing reward-side contributions: an ablation (§4.5) shows each is responsible for a ∼9-point aggregate drop across the three benchmarks when removed. Argument-value matching is a smaller-weight signal that disproportionately helps multi-turn dialog (τ^2).

Validity (R_{validity}). Each tool call is scored on three graduated levels, equally weighted: (1) the function name exists in the candidate schema; (2) all required parameters are present with compatible JSON types; (3) live execution against the MCP server succeeds without error. The validity reward averages these per-call scores across all N model calls. This graduated formulation follows Cheng et al. [2026], though we replace their AST-based check with live MCP schema validation and execution. It provides partial credit—a call with the correct name but wrong parameters receives ∼0.33; correct structure but failed execution receives ∼0.66—giving the model fine-grained learning signal even on partially-correct attempts.

Coverage (R_{coverage}). Measures whether the model's output covers all ground-truth workflow steps in the correct dependency order:

$$R_{\text{coverage}} = \frac{1}{|G|} \sum_{g \in G} m(g) \cdot o(g) \qquad (1)$$

where $m(g) = \mathbb{1}[g \text{ matched}]$ and $o(g) = \prod_{(j,g) \in E} \mathbb{1}[\mu(j) < \mu(g)]$ enforces dependency order; G is the set of ground-truth steps, E the dependency edges, and $\mu(\cdot)$ maps steps to their matched position in the model's output sequence. A model call matches a GT step only if it includes all of

the GT call's **argument keys**—preventing superficial matches where the model calls the right function with entirely wrong parameters.

Adaptive Efficiency ($R_{\text{efficiency}}$). Our key contribution. Coverage reward (pure recall) incentivizes verbosity: more tool calls increase the probability of matching GT steps, with no penalty for excess. We introduce an additive penalty with a *complexity-adaptive budget*:

$$R_{\text{efficiency}} = -\alpha \cdot \frac{\max(0,\ n_{\text{calls}} - B)}{B} \qquad (2)$$
$$B = n_{\text{gt}} + \lceil n_{\text{gt}} \cdot \beta \rceil$$

where n_{calls} is the number of model tool calls, n_{gt} is the total number of ground-truth tool calls (summed across all steps, including parallel calls within a single step), α controls penalty strength, and β controls the slack budget. Counting calls rather than steps is what makes the penalty consistent: a step with three parallel ground-truth calls budgets three calls, not one, so a model that emits the correct three is not penalized as if it had three excess calls.

The adaptive budget B is critical: a flat penalty ($B = n_{\text{gt}}$) over-penalizes complex tasks where exploration or information-gathering calls are legitimate. With $\beta = 0.5$, a 4-call task has budget $B = 6$, allowing 2 extra calls before any penalty applies. Key properties:

- *Complexity-adaptive*: Harder tasks (more GT steps) receive proportionally more slack.
- *Asymmetric*: Only penalizes *excess* calls (not missing calls, which are already captured by R_{coverage}).
- *GRPO-compatible*: Under group-relative advantage normalization, the penalty naturally scales with batch difficulty.

Tool Name Bonus (R_{name}). We observed in pilot training runs that tool-name selection is a primary early-training bottleneck: validity (R_{validity}) saturates well before the model reliably picks the right tools, leaving the coverage signal too sparse to drive consistent learning. We add an explicit reward that credits the model for calling tools whose names appear in the ground-truth set:

$$R_{\text{name}} = \frac{|\{c \in \hat{C} : c.\text{name} \in \text{GT\_names}\}|}{|\hat{C}|} \qquad (3)$$

where $\hat{C}$ is the set of model tool calls and GT_names is the set of tool names appearing in the ground truth. This is independent of ordering or argument matching—it purely rewards tool *selection* quality.

Argument-Value Matching (R_{arg}). Coverage matches a model call to a GT step using the function name and the set of *argument keys*—it does not compare the actual argument values, so a model that fills the correct slots with the wrong values still receives full coverage credit. To close this gap we add a value-matching bonus: for each pair of GT and model calls already aligned by name and key set, we score the fraction of argument values that match GT, and average across aligned pairs:

$$R_{\text{arg}} = \frac{1}{|M|} \sum_{(c,g)\in M} \frac{|\{k : c.\text{args}[k] = g.\text{args}[k]\}|}{|g.\text{args}|} \qquad (4)$$

where M is the set of (model call, GT call) pairs already aligned by Coverage. This signal is small in magnitude but consistently lifts BFCL Multi-Turn, τ^2-bench, and T-Eval at $w_{\text{arg}} = 0.1$; we did not observe similar gains at much higher or much lower weights, suggesting 0.1 is a sweet spot that nudges value choice without overshadowing the main reward terms.

Abstention Reward. For abstention examples ($G = \emptyset$): $R = 1.0$ if zero tool calls (correct abstention), $R = 0.0$ otherwise.

Combined Reward.

$$\begin{aligned} R_{\text{total}} = {} & w_{\text{val}}\, R_{\text{validity}} + w_{\text{cov}}\, R_{\text{coverage}} \\ & + w_{\text{eff}}\, R_{\text{efficiency}} + w_{\text{name}}\, R_{\text{name}} \\ & + w_{\text{arg}}\, R_{\text{arg}} \end{aligned} \qquad (5)$$

with $w_{\text{val}} = 0.5$, $w_{\text{cov}} = 0.5$, $w_{\text{eff}} = 0.15$, $w_{\text{name}} = 0.2$, $w_{\text{arg}} = 0.1$, $\alpha = 0.5$, and $\beta = 0.5$ throughout all experiments. The auxiliary weights (w_{eff}, w_{name}, w_{arg}) are kept smaller than the main components (w_{val}, w_{cov}) to avoid suppressing exploration early in training.

4 Experiments

4.1 Setup

Base Models. We evaluate on four instruction-tuned models spanning two architecture families: Qwen3-4B-Instruct[2], Qwen3-8B[3] [Qwen Team, 2025b], Qwen2.5-7B-Instruct[4] [Qwen Team, 2025a], and Granite-4.1-8B-Instruct[5] [IBM Research, 2025]. This covers 4B–8B parameters across Qwen and Granite architectures, testing whether a single reward configuration generalizes across model families.

Training. We train with Group Relative Policy Optimization [GRPO; Shao et al., 2024] via the VERL framework [Sheng et al., 2024] on 8×H100 GPUs. Key hyperparameters: batch size 16, rollout group size 16, tensor parallelism 2. Maximum prompt length is 12,384 tokens; maximum response length is 16,384 tokens. Multi-turn rollouts are capped at 5 user turns and 10 assistant turns. RL rollouts use 20 live MCP environments (see §3.1). Training uses 13,517 samples (10,895 multi-turn MCP conversations + 1,500 clarification + 806 When2Call + 316 xLAM-Irrelevance) for 350 GRPO steps in a single stage. KL coefficient is 0.01 for all models. Learning rate is the only per-family knob: Qwen3-4B uses 1×10^{-6}, Qwen3-8B uses 5×10^{-8}, Qwen2.5-7B uses 3×10^{-7}, and Granite-4.1-8B uses 3×10^{-7}. The Qwen2.5 and Granite families are more sensitive to learning rate than Qwen3-4B—at 1×10^{-6} both regress sharply on multi-turn benchmarks (see §4 discussion); we selected each model's LR from a small sweep over $\{1{\times}10^{-6}, 3{\times}10^{-7}, 5{\times}10^{-8}\}$. All models use identical reward configuration ($w_{\text{val}} = 0.5$, $w_{\text{cov}} = 0.5$, $w_{\text{eff}} = 0.15$, $w_{\text{name}} = 0.2$, $w_{\text{arg}} = 0.1$, $\alpha = 0.5$, $\beta = 0.5$).

Benchmarks. We evaluate on three benchmarks:

- **BFCL Multi-Turn** [Zhong et al., 2025]: Evaluates multi-step function calling with subcategories for base multi-turn, missing function, missing parameter, and long context scenarios. We report Overall MT accuracy and all subcategories.
- **τ^2-bench** [Barres et al., 2025]: Evaluates conversational agents in a dual-control environment where both agent and user can modify shared state across airline, retail, and telecom domains. An LLM user simulator drives multi-turn conversations where the agent must call tools to resolve requests. We report per-domain and average task completion rates.
- **T-Eval** [Chen et al., 2023]: Evaluates tool-use capabilities across six dimensions: instruction following, planning, reasoning, retrieval, understanding, and review. We report scores averaged over JSON and string prompt formats.

For τ^2-bench, which requires an LLM user simulator to drive multi-turn interactions, we use `Qwen3-VL-235B-A22B-Instruct`[6] as the simulator across all runs to keep evaluations directly comparable.

4.2 Main Results

Table 3 presents our main results comparing base models against our method on BFCL Multi-Turn and τ^2-bench.

BFCL Multi-Turn. All four models improve on BFCL MT Overall: Qwen3-4B (+10.2), Qwen2.5-7B (+3.7), Qwen3-8B (+0.9), and Granite-4.1-8B (+0.1, essentially flat). The MT Base subcategory shows the strongest gain on Qwen3-4B (+12.0), indicating substantially improved multi-step orchestration; Granite-4.1-8B's near-zero overall change masks a +6.5 gain on Multi-Turn Miss-Function offset by smaller regressions on Base (-1.5) and Long-Context (-2.5).

[2] `https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507`

[3] `https://huggingface.co/Qwen/Qwen3-8B`

[4] `https://huggingface.co/Qwen/Qwen2.5-7B-Instruct`

[5] `https://huggingface.co/ibm-granite/granite-4.1-8b-instruct`

[6] `https://huggingface.co/Qwen/Qwen3-VL-235B-A22B-Instruct`

Table 3: Main results across model families. All models trained for 350 GRPO steps with PROVE.

Model	Method	BFCL Multi-Turn					τ^2-bench			
		Base	Miss Param	Miss Func	Long Ctx	Overall	Airline	Retail	Telecom	Avg
Qwen3-4B	Base	23.5	14.5	14.0	23.0	18.8	26.8	35.1	13.2	25.0
	PROVE	**35.5**	**18.5**	**29.0**	**33.0**	**29.0**	**28.0**	**42.1**	**15.8**	**28.6**
	Δ	+12.0	+4.0	+15.0	+10.0	+10.2	+1.2	+7.0	+2.6	+3.6
Qwen3-8B	Base	30.5	19.5	31.5	**22.5**	26.0	24.0	28.9	**24.6**	25.8
	PROVE	**34.0**	**21.5**	**32.0**	20.0	**26.9**	**30.0**	**34.2**	23.7	**29.3**
	Δ	+3.5	+2.0	+0.5	-2.5	+0.9	+6.0	+5.3	-0.9	+3.5
Qwen2.5-7B	Base	17.0	10.0	14.5	11.0	13.1	16.0	12.3	16.7	15.0
	PROVE	**22.5**	**13.5**	**18.0**	**13.0**	**16.8**	**26.0**	**16.7**	**22.8**	**21.8**
	Δ	+5.5	+3.5	+3.5	+2.0	+3.7	+10.0	+4.4	+6.1	+6.8
Granite-4.1-8B	Base	**44.5**	**26.0**	23.5	**39.5**	33.4	8.0	**49.1**	17.5	24.9
	PROVE	43.0	24.0	**30.0**	37.0	**33.5**	**22.0**	47.4	**24.6**	**31.3**
	Δ	-1.5	-2.0	+6.5	-2.5	+0.1	+14.0	-1.8	+7.0	+6.4

τ^2-bench. All four models improve: Qwen2.5-7B (+6.8), Granite-4.1-8B (+6.4), Qwen3-4B (+3.6), and Qwen3-8B (+3.5). Granite-4.1-8B's gain is concentrated on the airline domain (+14.0, from 8.0 to 22.0) and telecom (+7.0); Qwen2.5-7B's gain is the most uniform across the three domains, with all three improving by $+4$ pts or more. The earlier divergence pattern we observed for Qwen3-8B at the launcher's default learning rate (1×10^{-6}) disappears at the lower 5×10^{-8} used here—the model stabilizes and posts a TAU2 gain comparable to Qwen3-4B.

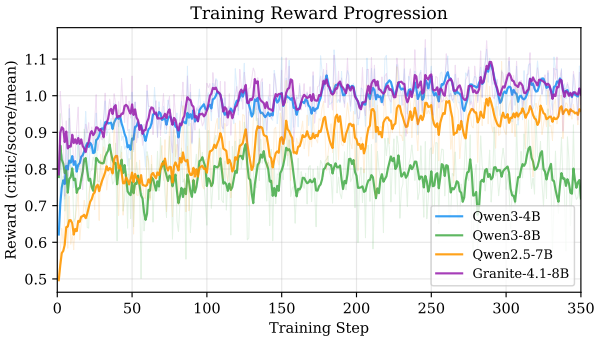

Figure 3: Training reward (critic/score/mean) over 350 GRPO steps. All models converge smoothly without instability.

4.3 T-Eval Results

Table 4 reports T-Eval scores across all four models.

Analysis. All four models improve on T-Eval overall: Qwen2.5-7B (+6.5), Granite-4.1-8B (+4.4), Qwen3-8B (+1.9), and Qwen3-4B (+1.5). Qwen2.5-7B shows the strongest gains across five of six dimensions; Granite-4.1-8B posts the single largest improvement on review (+15.6). The retrieve and plan dimensions benefit most consistently across models, as these directly exercise the multi-step tool-dependency skills our reward targets.

4.4 Training Dynamics

Figure 3 shows the training reward progression over 350 steps. Three of four models exhibit rapid initial improvement (steps 0–50) followed by gradual refinement, confirming that the reward provides strong learning signal from the start; Qwen3-8B is comparatively flat throughout.

Figure 4 decomposes the reward into its five components at steps 0, 150, and 350 across all models. Across Qwen3-4B, Qwen2.5-7B, and Granite-4.1-8B, the four trainable components move in the expected directions: R_{cov} shows the largest gains, R_{val} saturates early (most improvement by step 150), R_{eff} shrinks toward zero as models learn conciseness, and R_{name} improves steadily. Qwen3-8B is the exception: every component is essentially flat throughout training, consistent with the conservative learning rate required to stabilize this base model and with its smaller eval-time gains.

Table 4: T-Eval results (0–100). Scores averaged over JSON and string prompt formats. All models trained for 350 steps with PROVE.

Model	Method	Instruct	Plan	Reason	Retrieve	Understand	Review	**Overall**
Qwen3-4B	Base	95.5	66.4	62.8	87.5	77.3	**70.8**	76.7
Qwen3-4B	PROVE	**98.5**	**68.7**	**63.7**	**90.6**	**78.7**	68.8	**78.2**
	Δ	+3.0	+2.3	+1.0	+3.1	+1.4	-2.1	+1.5
Qwen3-8B	Base	**98.8**	64.7	**64.2**	88.0	77.8	79.2	78.8
Qwen3-8B	PROVE	94.8	**65.2**	63.0	**92.7**	**81.8**	**86.5**	**80.7**
	Δ	-4.1	+0.4	-1.2	+4.7	+4.0	+7.3	+1.9
Qwen2.5-7B	Base	73.4	54.1	57.7	79.2	62.2	75.0	66.9
Qwen2.5-7B	PROVE	**81.8**	**60.4**	**63.1**	**88.0**	**69.7**	**77.1**	**73.4**
	Δ	+8.4	+6.3	+5.5	+8.9	+7.6	+2.1	+6.5
Granite-4.1-8B	Base	93.5	68.2	63.4	87.0	74.0	55.2	73.5
Granite-4.1-8B	PROVE	**94.7**	**69.7**	**63.7**	**91.7**	**76.9**	**70.8**	**77.9**
	Δ	+1.1	+1.5	+0.4	+4.7	+2.9	+15.6	+4.4

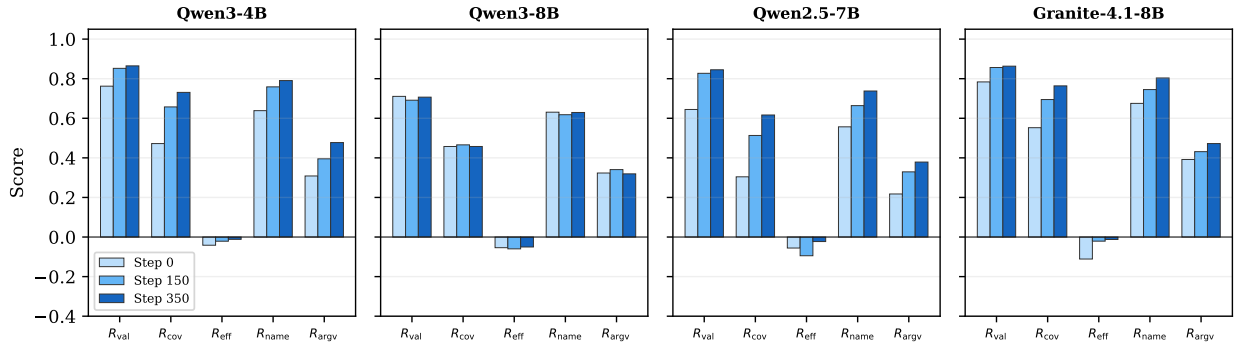

Figure 4: Reward component progression at steps 0, 150, and 350 across all four models. Coverage (R_{cov}) shows the largest and most consistent improvement; validity (R_{val}) saturates early.

4.5 Reward-Component Ablation

To verify that each reward term contributes, we retrain Qwen3-4B with the paper recipe (350 GRPO steps) under six variants: each of the five reward components zeroed in turn, plus a flat-budget variant that disables the adaptive efficiency target ($\beta = 0$). Table 5 reports BFCL Multi-Turn, τ^2-bench average, and T-Eval Overall against the full reward.

Every component carries weight on at least one benchmark. Removing R_{name} produces the largest broad-spectrum drop (BFCL MT -6.0, T-Eval -2.4); removing R_{arg} collapses τ^2 (-4.2 pts) while paradoxically lifting MT (the model relearns to fire calls aggressively without the argument-quality penalty). Dropping R_{validity} or R_{coverage} each costs MT -2 to -5 pts together with T-Eval drops, confirming both terms shape generalisation beyond their specific signals. The strongest single result is the $\beta = 0$ row: flattening the adaptive budget in $R_{\text{efficiency}}$ loses MT -6.6 and τ^2 -3.1, isolating the per-task ground-truth-call target as the most load-bearing piece of the efficiency formulation.

4.6 Comparison to Supervised Fine-Tuning

To isolate the contribution of the reward signal, we train an SFT baseline on the same training data (3 epochs, ms-swift) for each model and evaluate alongside the base model and PROVE. Table 6 reports BFCL Multi-Turn, τ^2-bench, and T-Eval Overall.

PROVE dominates SFT on 11 of 12 cells (4 models $\times$ 3 benchmarks), only ceding BFCL MT on Qwen2.5-7B. The most striking pattern is on τ^2-bench: SFT regresses below the base model on 3/4 models (Qwen3-4B -5.4, Qwen3-8B -14.3, Granite-4.1-8B -5.3 pts), suggesting that imitating the synthetic trajectories without execution-

Table 5: Reward-component ablation on Qwen3-4B with the paper recipe (350 GRPO steps). Each row drops one term from the combined reward; the last row keeps all terms but flattens the adaptive efficiency budget ($\beta = 0$). Δ vs. the full reward is shown in parentheses. TAU2 is reported on a 0–100 scale. Code-side names map as $R_{\text{validity}} \equiv R_{\text{atomic}}$, $R_{\text{coverage}} \equiv R_{\text{orch}}$, $R_{\text{efficiency}} \equiv R_{\text{eff}}$.

Variant	BFCL MT	TAU2	T-Eval
Full reward (PROVE)	**29.0**	**28.6**	**78.2**
$-R_{\text{validity}}$	23.8 -5.2	30.7 +2.1	74.8 -3.4
$-R_{\text{coverage}}$	26.6 -2.4	29.0 +0.4	76.7 -1.5
$-R_{\text{efficiency}}$	25.0 -4.0	28.5 -0.1	76.5 -1.6
$-R_{\text{name}}$	23.0 -6.0	27.8 -0.8	75.8 -2.4
$-R_{\text{arg}}$	30.6 +1.6	24.5 -4.2	77.6 -0.6
$R_{\text{efficiency}}$ flat ($\beta = 0$)	22.4 -6.6	25.5 -3.1	78.6 +0.4

grounded reward damages the multi-turn agentic behaviour the base model already had. PROVE never regresses on τ^2 relative to base, and improves it by $+3.5$ to $+6.8$ across all four models. The same effect is visible but milder on T-Eval, where SFT trails PROVE by 1.2–4.4 pts. This indicates that the data alone is insufficient: the reward signal is what converts the trajectories into reliable multi-step orchestration.

4.7 Teacher Ablation

The orchestrator and clarification pipelines (§3.2) use a teacher LLM to drive query generation and assistant turns. To check whether our results depend on a particular teacher, we regenerate the full training mixture using Qwen3-32B as the teacher in place of Gemma-4-31B-it, keeping every other knob fixed (same prompts, same $\binom{n}{2}$ dependency-graph cache, same filtering). We then train Qwen3-4B and Qwen3-8B with the paper recipe (350 steps, $R_{\text{arg}} = 0.1$, family-specific LR).

Results in Table 7 are within noise on all three benchmarks: τ^2-bench scores are essentially identical across teachers ($\Delta < 1$ pt for both models) and T-Eval differs by at most 1.4 pts. BFCL Multi-Turn shows a Qwen3-4B regression with the Qwen3-32B teacher (-6.4 pts) but no such effect on Qwen3-8B ($+0.2$). This robustness to teacher choice suggests the method's gains stem from the reward formulation and the live-execution training loop rather than a particular teacher's writing style.

Table 6: Comparing PROVE (RL with our reward) against SFT on the same training data and the base model. SFT is trained for 3 epochs with ms-swift; PROVE runs 350 GRPO steps. PROVE dominates SFT on all three benchmarks for 3/4 models, and ties or wins on the fourth. Best result per row in bold. τ^2 on a 0–100 scale.

Model	Method	BFCL MT	τ^2	T-Eval
Qwen3-4B	Base	18.8	25.0	76.7
	SFT	26.2	19.6	74.7
	PROVE	**29.0**	**28.6**	**78.2**
Qwen3-8B	Base	26.0	25.8	78.8
	SFT	22.4	11.5	76.3
	PROVE	**26.9**	**29.3**	**80.7**
Qwen2.5-7B	Base	13.1	15.0	66.9
	SFT	**22.5**	15.2	72.2
	PROVE	16.8	**21.8**	**73.4**
Granite-4.1-8B	Base	33.4	24.9	73.5
	SFT	29.0	19.6	75.2
	PROVE	**33.5**	**31.3**	**77.9**

Table 7: Teacher ablation. We re-run the data-synthesis pipeline (orchestrator + clarification) with Qwen3-32B as the teacher in place of Gemma-4-31B-it, then train Qwen3-4B and Qwen3-8B with the paper recipe (350 GRPO steps). Results are within noise across both teachers on all three benchmarks, and both improve over the base model. Best result per row in bold; τ^2 on a 0–100 scale.

Model	Setting	BFCL MT	τ^2	T-Eval
Qwen3-4B	Base	18.8	25.0	76.7
	Gemma-31B	**29.0**	28.6	**78.2**
	Qwen3-32B	22.6	**29.0**	76.8
Qwen3-8B	Base	26.0	25.8	78.8
	Gemma-31B	26.9	29.3	**80.7**
	Qwen3-32B	**27.1**	**29.7**	79.8

5 Conclusion

We presented PROVE (Programmatic Rewards On Verified Environments), which couples live state-

ful MCP environments, state-grounded multi-turn data synthesis, and a programmatic multi-component RL reward into a single training loop—no LLM-as-judge, no manual annotation. Two reward-side components carry the most weight: an adaptive efficiency penalty $R_{\text{efficiency}}$ with a complexity-scaled call budget, which counters the verbosity incentive of pure-recall coverage rewards, and a tool-name signal R_{name}; an ablation (§4.5) shows each is responsible for a $\sim$9-point aggregate drop across the three benchmarks when removed. Validity, coverage, and an argument-value matching signal (the latter disproportionately helping multi-turn dialog) round out the five-component reward. With a single reward configuration and only learning rate tuned per family, four models from two architecture families improve on BFCL Multi-Turn, τ^2-bench, and T-Eval, supporting our central claim: a compact, fully programmatic reward coupled with grounded data synthesis can drive consistent multi-step tool-orchestration gains without large-scale data pipelines or expensive judge models.

References

Anthropic. Model context protocol. 2024. https://modelcontextprotocol.io/.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment, 2025.

Cheng Chen et al. ToolRL: Reward is all tool learning needs, 2025.

Zehui Chen, Weihua Du, Wenwei Zhang, Kuikun Liu, Jiangning Liu, Miao Zheng, Jingming Zhuo, Songyang Zhang, Dahua Lin, Kai Chen, and Feng Zhao. T-Eval: Evaluating the tool utilization capability of large language models step by step, 2023.

Jiayang Cheng, Xin Liu, Zhihan Zhang, Haoyang Wen, Zixuan Zhang, Qingyu Yin, Shiyang Li, Priyanka Nigam, Bing Yin, Chao Zhang, and Yangqiu Song. Training LLMs for multi-step tool orchestration with constrained data synthesis and graduated rewards, 2026.

Maxwell Crouse, Ibrahim Abdelaziz, Kshitij Fadnis, Siva Sankalp Patel, Kinjal Basu, Chulaka Gunasekara, Sadhana Kumaravel, Asim Munawar, and Pavan Kapanipathi. Simulating complex multi-turn tool calling interactions in stateless execution environments, 2026.

Jiazhan Feng, Ruochen Li, Junheng Hao, and Xin Eric Wang. ReTool: Reinforcement learning for strategic tool use in LLMs, 2025.

IBM Research. Granite 4.1 language models, 2025. `https://huggingface.co/ibm-granite/granite-4.1-8b-instruct`.

MadeAgents. xLAM-Irrelevance-7.5k. `https://huggingface.co/datasets/MadeAgents/xlam-irrelevance-7.5k`, 2024. HuggingFace dataset.

Shishir G. Patil, Huanzhi Mao, Charlie Cheng-Jie Ji, Fanjia Yan, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Advances in Neural Information Processing Systems*, 2024.

Akshara Prabhakar, Zuxin Liu, Weiran Yao, Jianguo Zhang, Tian Lan, Rithesh Murthy, Zhiwei Liu, Thai Hoang, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. APIGen-MT: Agentic PIpeline GENeration for multi-turn tool-use agents, 2025.

Qwen Team. Qwen2.5 technical report, 2025a.

Qwen Team. Qwen3 technical report, 2025b.

Qwen Team. AgenticQwen: Building tool-use agents with dual data flywheels and behavior trees, 2026.

Hayley Ross, Ameya Sunil Mahabaleshwarkar, and Yoshi Suhara. When2Call: When (not) to call tools. In *Proceedings of NAACL*, 2025.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024.

Guangming Sheng et al. VERL: An extensible framework for post-training of LLMs via RL, 2024. https://github.com/volcengine/verl.

Zhangchen Xu, Adriana Meza Soria, Shawn Tan, Anurag Roy, Ashish Sunil Agrawal, Radha Poovendran, and Rameswar Panda. TOUCAN: Synthesizing 1.5m tool-agentic data from real-world MCP environments, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023.

Fan Yin, Zifeng Wang, I-Hung Hsu, Jun Yan, Ke Jiang, Yanfei Chen, Jindong Gu, Long T. Le, Kai-Wei Chang, Chen-Yu Lee, Hamid Palangi, and Tomas Pfister. Magnet: Multi-turn tool-use data synthesis and distillation via graph translation, 2025.

Yuanqing Yu, Zhefan Wang, Weizhi Ma, Zhicheng Guo, Jingtao Zhan, Shuai Wang, Chuhan Wu, Zhiqiang Guo, and Min Zhang. StepTool: Enhancing multi-step tool usage in LLMs via step-grained reinforcement learning, 2024.

Xingshan Zeng, Weiwen Liu, Lingzhi Wang, Liangyou Li, Fei Mi, Yasheng Wang, Lifeng Shang, Xin Jiang, and Qun Liu. ToolACE-MT: Non-autoregressive generation for agentic multi-turn interaction. In *International Conference on Learning Representations (ICLR)*, 2026.

Yirong Zeng, Xiao Ding, Yutai Hou, Yuxian Wang, Li Du, Juyi Dai, et al. Tool Zero: Training tool-augmented LLMs via pure RL from scratch. In *Findings of EMNLP*, 2025.

Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems, 2024.

Shaokun Zhang, Yi Dong, Jieyu Zhang, Jan Kautz, Bryan Catanzaro, Andrew Tao, Qingyun Wu, Zhiding Yu, and Guilin Liu. Nemotron-Research-Tool-N1: Exploring tool-using language models with reinforced reasoning, 2025a.

Yabo Zhang, Yirong Zeng, Qingfu Li, Zhenyi Hu, Kai Han, and Wangmeng Zuo. Tool-R1: Sample-efficient reinforcement learning for agentic tool use, 2025b.

Weikang Zhao, Xili Wang, et al. MUA-RL: Multi-turn user-interacting agent reinforcement learning for agentic tool use, 2025.

Lucen Zhong, Zhengxiao Du, Akari Asai, and Jie Tang. ComplexFuncBench: Exploring multi-step and constrained function calling under long-context scenario, 2025.